DECK 1 • SLL FOUNDATIONS

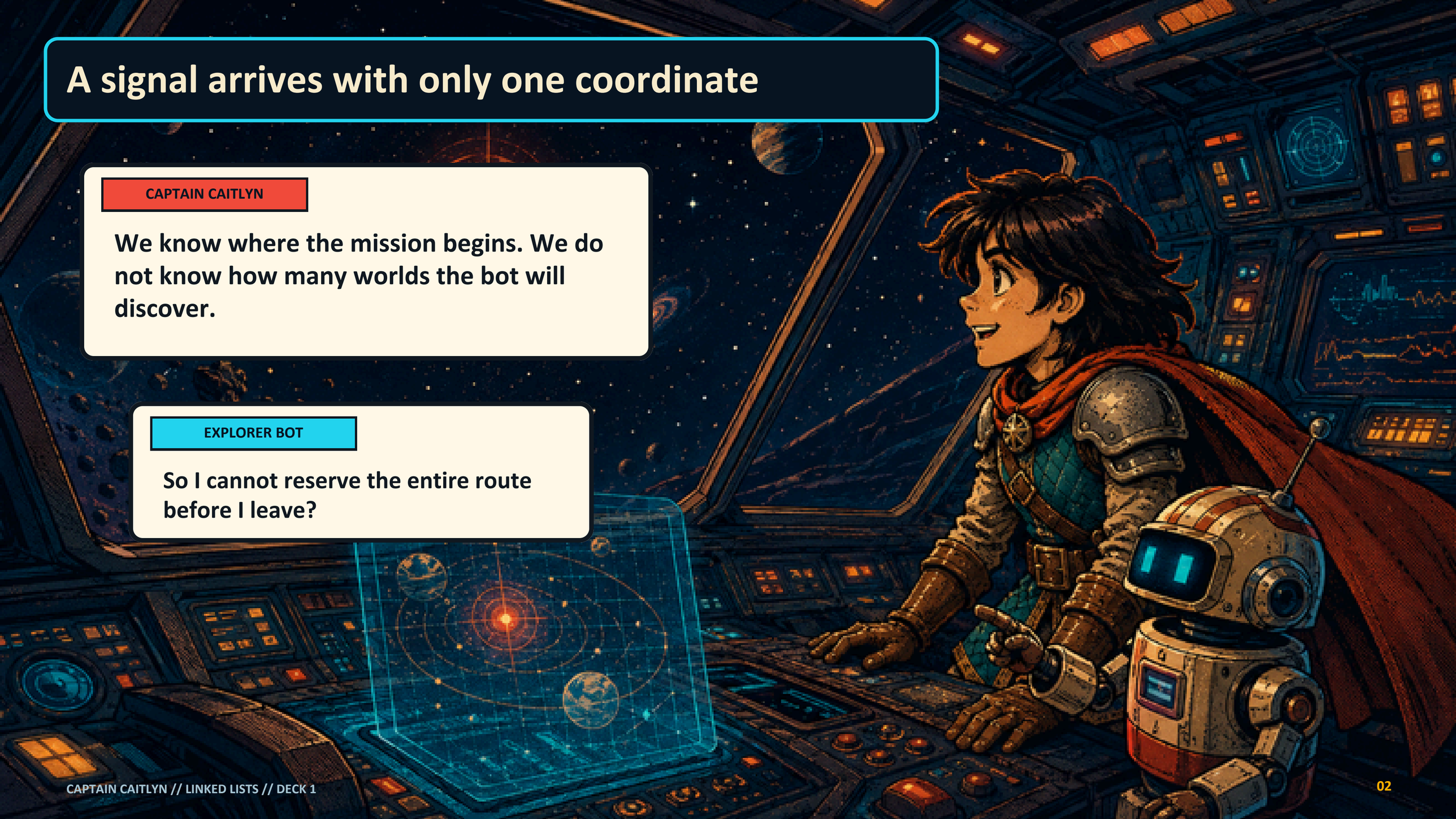
BEYOND THE SIGNAL

Why linked lists exist and how a chain of addresses creates order

Captain Caitlyn • Explorer Bot • A trail of addresses

Lijo M. Jose

A signal arrives with only one coordinate



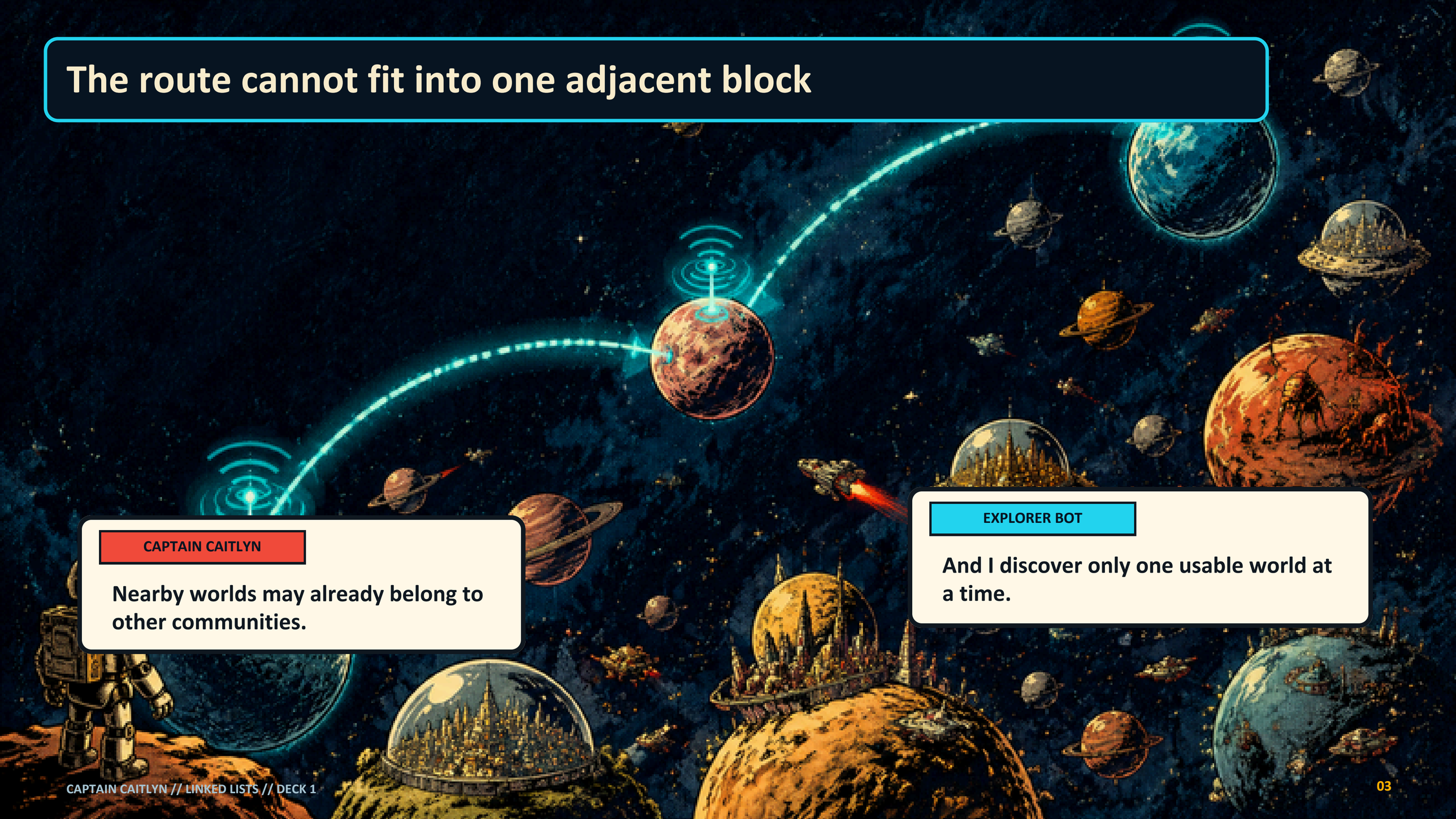
CAPTAIN CAITLYN

We know where the mission begins. We do not know how many worlds the bot will discover.

EXPLORER BOT

So I cannot reserve the entire route before I leave?

The route cannot fit into one adjacent block



CAPTAIN CAITLYN

Nearby worlds may already belong to other communities.

EXPLORER BOT

And I discover only one usable world at a time.

RECALL

Arrays are fast because every element has a predictable neighbour

address = base + index × element size

The formula works because the cells are contiguous and equally sized.

Scattered space needs a different kind of order

ARRAY • POSITION CREATES ORDER

LINKED LIST • ADDRESS CREATES ORDER

ASK THE CLASS

What must the bot leave behind on every discovered planet?

A Only the information collected on this planet

B Only the coordinates of the next planet

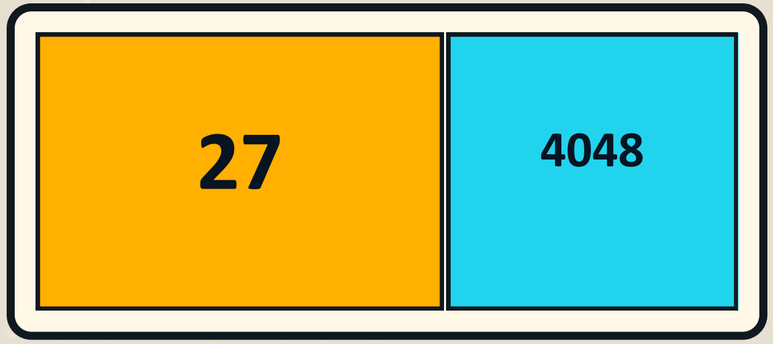
C The information and the next planet's address

PAUSE • ASK THEM TO DEFEND THE CHOICE

REVEAL

One beacon needs two compartments

@6200



EXPLORER BOT

The first compartment stores what I found. The second stores where the route continues.

The beacon itself may be stored anywhere.

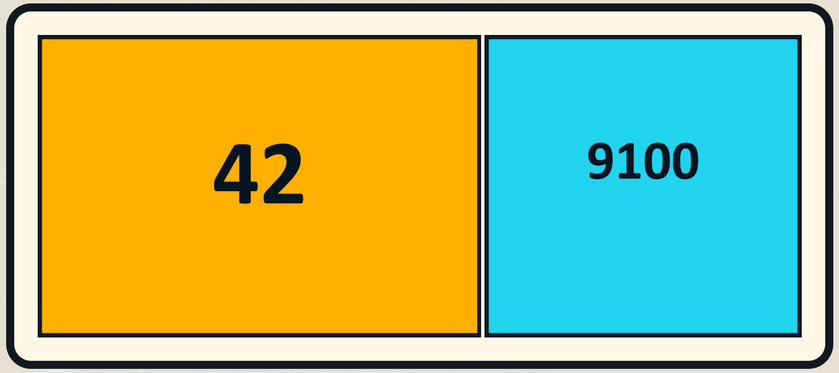
CONCEPT MAP

The story translates directly into memory

SPACE STORY		PROGRAM MEMORY
planet	→	allocated memory location
beacon	→	node
observations	→	data field
next coordinates	→	next pointer
mission's first coordinate	→	head pointer

A node is data plus a link

@4048



payload

address of another node

The pointer does not contain the next node. It contains its address.

NODE = DATA + LINK

The C structure says exactly the same thing

NODE DEFINITION

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

data

stores the payload

next

stores another Node's address

ASK BEFORE REVEALING

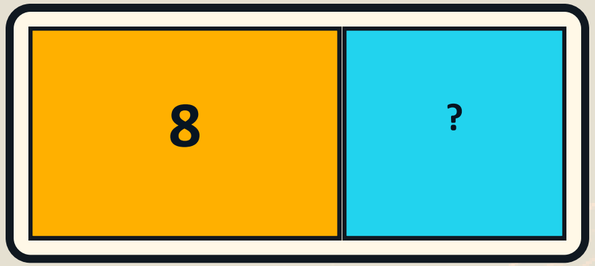
Why is next declared as struct Node *?

next must eventually hold…

A another integer

B the address of another Node

@6200



What type of pointer can legally hold that address?

REVEAL

A pointer's type describes the object found at that address

```
struct Node *next;
```

Read it as: next is a pointer to a struct Node

The structure may contain a pointer to its own type even though it cannot contain another complete Node inside itself.

malloc asks for one free node-sized location

MEMORY ALLOCATOR

Any suitable free platform will do. It does not have to neighbour the previous node.

DYNAMIC • ONE NODE AT A TIME

Create one node—and check whether allocation succeeded

ALLOCATE SAFELY

```
Node *newNode = malloc(sizeof *newNode);

if (newNode == NULL) {
    /* allocation failed */
    return;
}
```

Why sizeof *newNode?

It automatically matches the object that newNode points to.

No cast is required in C.

A fresh node does not yet know its successor

EXPLORER BOT

Store the planet data. Set next to NULL. Only then is the new beacon in a safe standalone state.

INITIALISE

```
newNode->data = 27;
newNode->next = NULL;
```

Head stores the only coordinate needed to enter the chain

Node *head = (address of first node);

The smallest lists reveal the invariants

The Bot hasn’t started the exploration

EMPTY LIST

head → NULL

Empty: head == NULL

The Bot finished exploration of planet 1

ONE-NODE LIST

One node: head->next == NULL

ASK THE CLASS

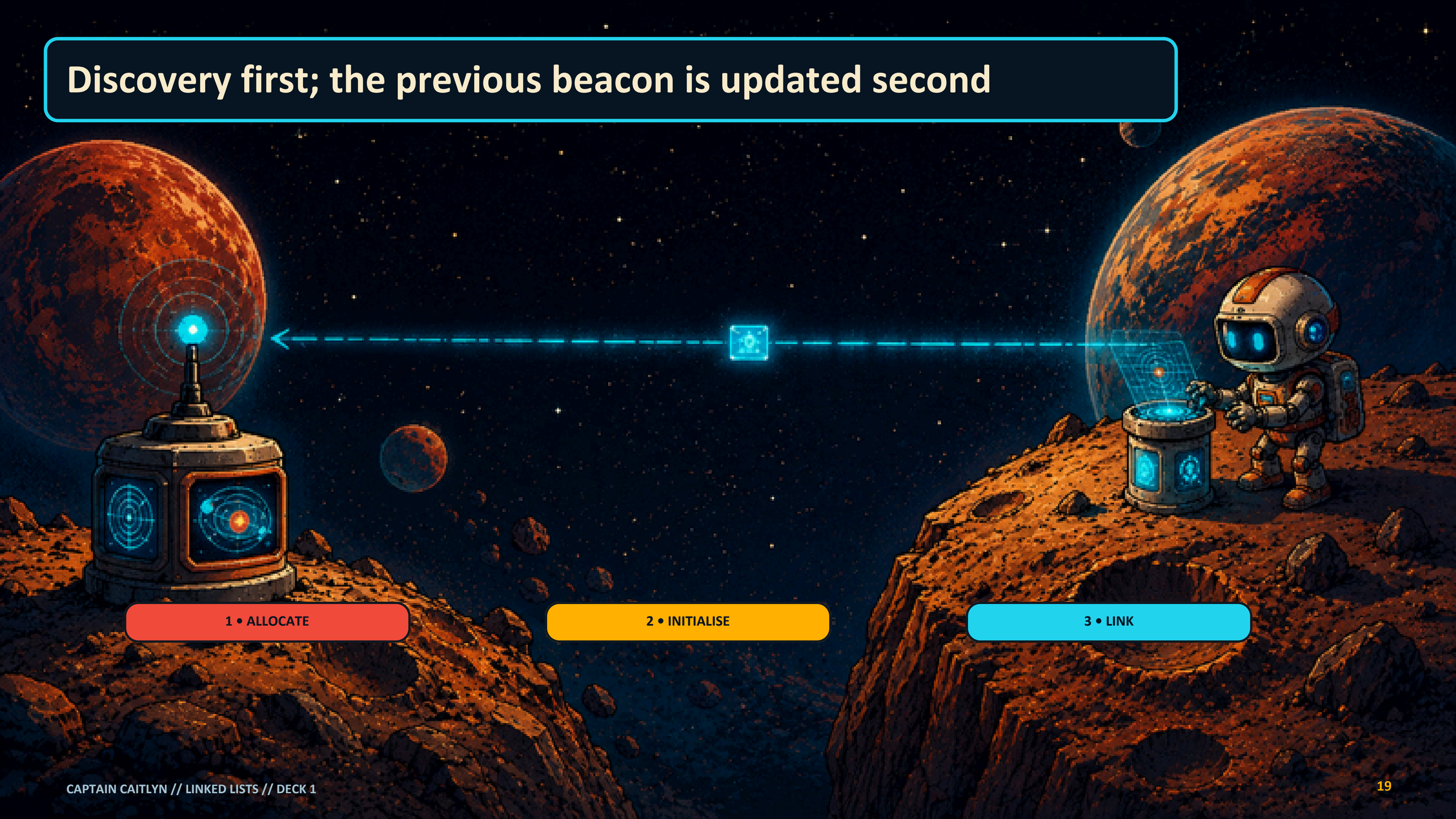
When is it safe to update the previous beacon?

A Before the new node has been allocated

B After allocation, but before setting its fields

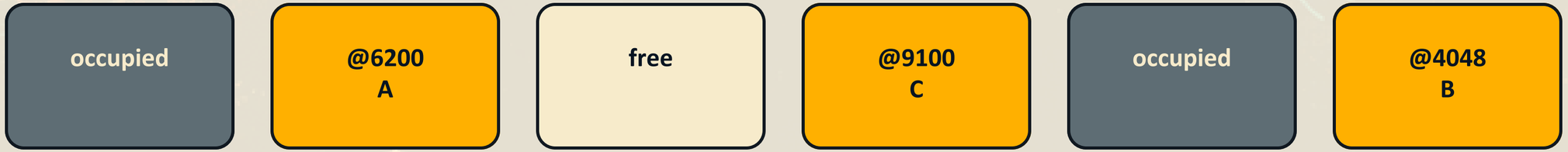
C After the new node exists and is safely initialised

Discovery first; the previous beacon is updated second

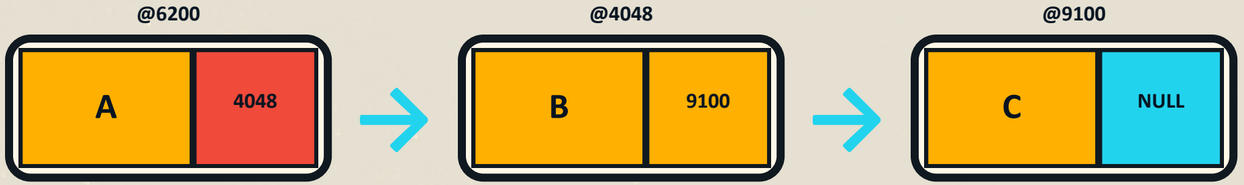


Physical address order is not the list order

MEMORY



LOGICAL CHAIN



6200 → 4048 → 9100

Now we can name the structure

SINGLY LINKED LIST

A sequence of nodes whose logical order is stored using one-way links.

chain — **is the connected sequence formed by following next from head**

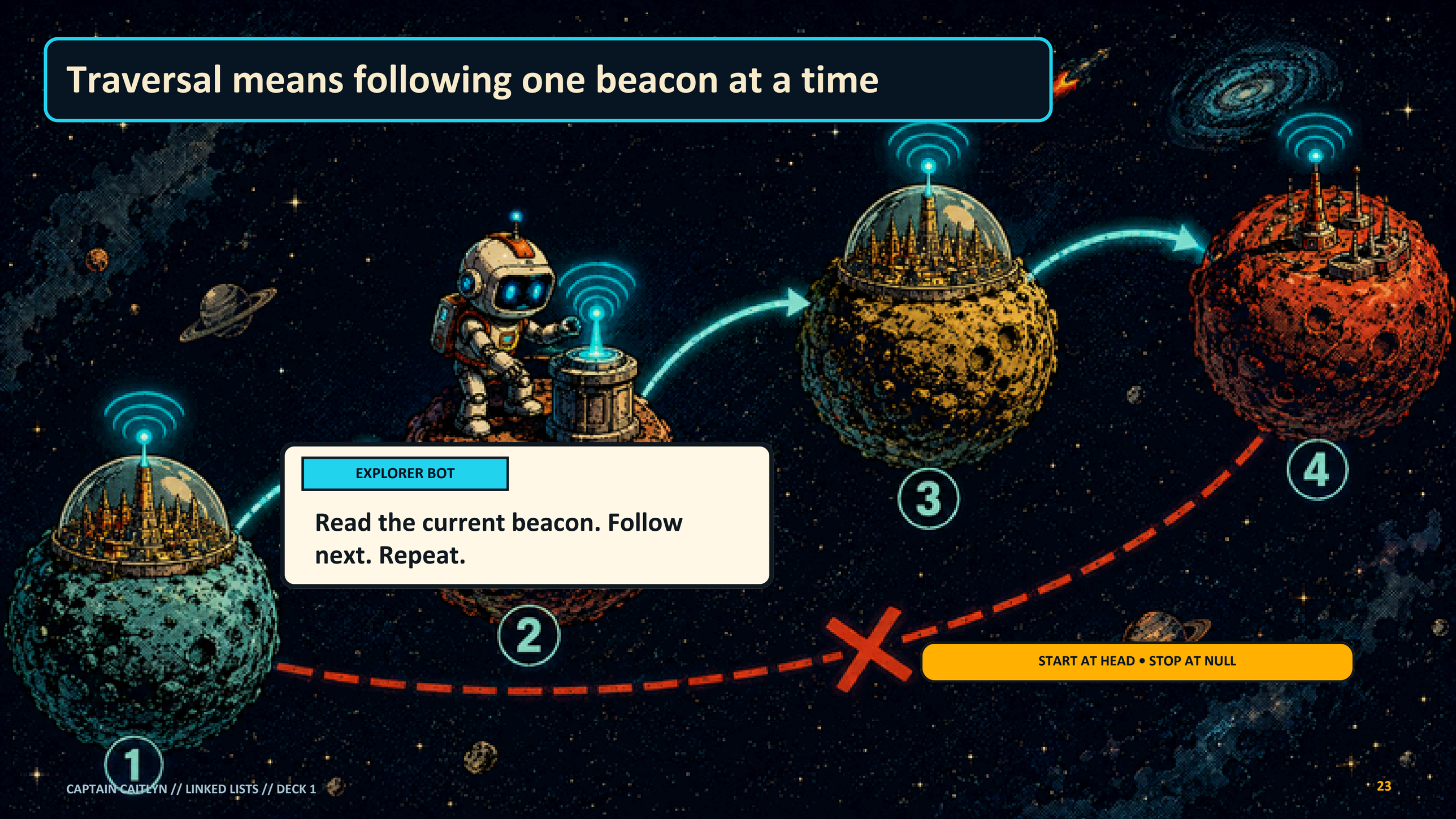
Checkpoint: can you reconstruct the route from memory?

Without looking back, draw:

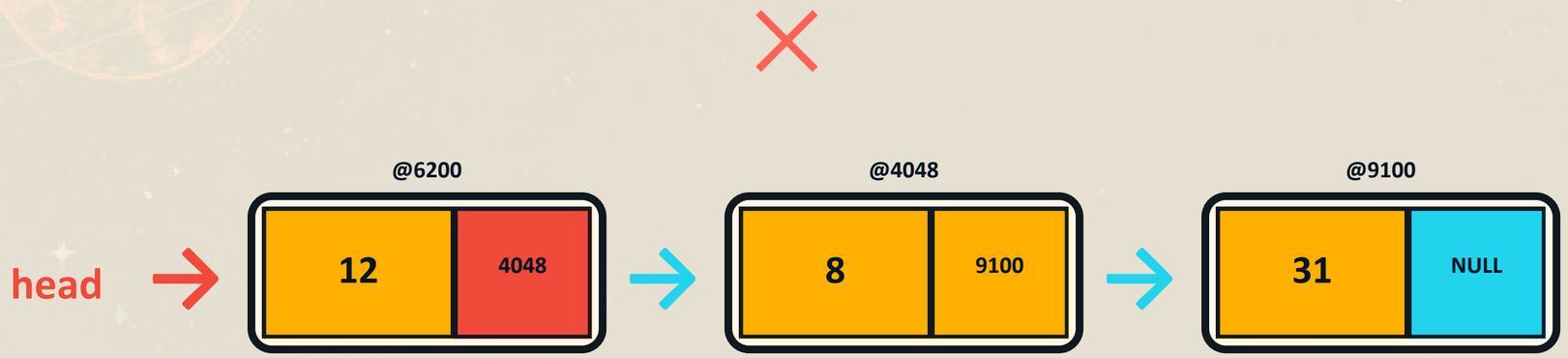
- a Node with data and next
- an empty list
- a three-node list whose addresses are not sorted

Then explain where traversal begins and ends.

Traversal means following one beacon at a time



There is no legal shortcut to the third node



head + 2 is not a linked-list operation

The traversal loop mirrors the bot's movement

TRAVERSE

```
Node *temp = head;

while (temp != NULL) {
    /* use temp->data */
    temp = temp->next;
}
```

temp moves

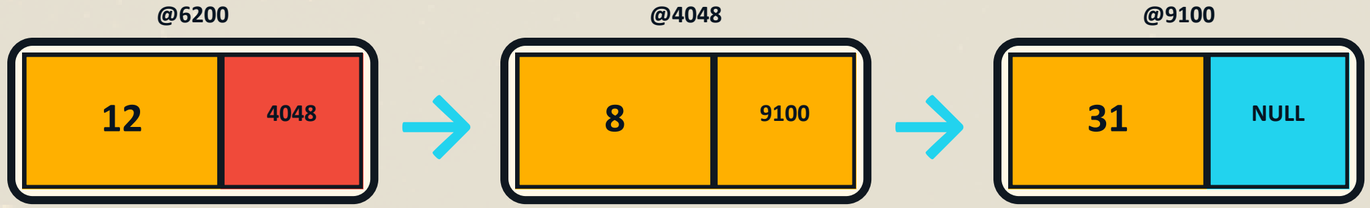
The nodes do not.

head remains unchanged.

CLASS CHALLENGE

Trace temp without sorting the addresses

head = 6200



YOUR TURN

1. What prints?
2. Which addresses does temp hold?
3. Where does temp finish?

PAUSE BEFORE REVEAL

CHALLENGE REVEAL

The links (not the addresses) decide the order

OUTPUT	12 8 31
temp path	6200 → 4048 → 9100 → NULL
final temp	NULL

Address order is irrelevant.

Display is traversal with a printf at every node

DISPLAY

```
void display(const Node *head) {
    const Node *temp = head;

    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

12 → 8 → 31 → NULL

const promises that display will not modify the nodes.

ASK THE CLASS

Which condition correctly visits every node exactly once?

A `while (temp->next != NULL)`

B `while (temp != NULL)`

C `while (temp == head)`

Which version accidentally skips the last node?

Search is traversal that stops when the data matches

EXPLORER BOT

Check one beacon, then follow next. The target may be first, last or absent.

BEST CASE • FIRST NODE

WORST CASE • LAST OR ABSENT

Search returns the address of the matching node

SEARCH

```
Node *search(Node *head, int key) {
    Node *temp = head;

    while (temp != NULL) {
        if (temp->data == key)
            return temp;
        temp = temp->next;
    }
    return NULL;
}
```

found

return the node's address

not found

return NULL

QUICK CHALLENGE

Predict the number of comparisons

LIST

4 → 9 → 2 → 7 → NULL

search 4 ?

search 7 ?

search 5 ?

Best case: 1

Worst case: n

COMPLEXITY MAP

The cost follows the number of links we must cross

OPERATION	TIME	WHY
read head	O(1)	the entry address is stored
visit every node	O(n)	each link is followed once
access the i-th node	O(n)	no direct indexing
search by value	O(n)	the target may be last or absent

TRADE-OFF

Linked lists buy flexible growth by giving up direct access

WHAT WE GAIN

- Nodes may live anywhere
- Size grows one node at a time
- Known links can be rewired locally

WHAT WE PAY

- One pointer per node
- No O(1) indexing
- Broken links can lose the route

100

BUG RADAR

Four small mistakes can destroy the chain

UNINITIALISED next

may point anywhere

LOST head

the whole chain becomes unreachable

NULL dereference

reading temp->data after temp is NULL

FORGOTTEN free

removed memory remains allocated

EXIT TICKET

Mission log: order can exist without adjacency

NODE	HEAD	TRAVERSAL	END
data + next	first address	follow next	next == NULL

CAPTAIN CAITLYN

Next mission: add and remove planets without losing the remaining route.

Before leaving: explain why address 9000 may legally come before address 2000.

DECK 2 • REWIRING THE ROUTE